

桌面宠物 — Unity 迁移方案设计文档

2026 年 6 月 11 日

摘要

本文档为桌面宠物项目从 GDI+ 原生方案迁移至 Unity 3D 引擎的技术方案设计。新方案将利用 Unity 引擎的 3D 渲染能力直接加载米哈游（崩坏：星穹铁道）的角色 3D 模型，结合透明窗口叠加技术实现桌面宠物效果。本文档涵盖项目目标、技术架构、实施路线和关键脚本设计，作为后续实际开发的指导文档。

目录

1	项目概述	3
1.1	项目背景	3
1.2	项目目标	3
1.3	开发环境	3
2	新旧架构对比	4
3	技术架构	4
3.1	整体架构	4
3.2	透明窗口实现	5
3.3	模型渲染管线	5
3.4	进程间通信（可选）	6
4	关键脚本设计	6
4.1	DesktopPet.cs — 宠物主控	6
4.2	DragHandler.cs — 拖拽交互	8
4.3	WindowOverlay.cs — 透明窗口管理	10
4.4	TouchController.cs — 点击反馈	12
5	行走动画设计原理	13
5.1	核心哲学	13
5.2	四大关键姿势	14
5.3	垂直弹跳相位	14
5.4	手臂与腿的交叉对位	14
5.5	身体前倾与速度	15
5.6	头部运动	15

目录	2
5.7 Live2D 参数映射	15
5.8 常见错误	16
6 实施路线	16
6.1 第一阶段：环境搭建（第 1 天）	17
6.2 第二阶段：模型导入（第 2-3 天）	17
6.3 第三阶段：交互实现（第 4-6 天）	17
6.4 第四阶段：集成与优化（第 7-10 天）	17
7 模型获取方案	18
8 Live2D 过渡预留	18
9 目录结构规划	19

1 项目概述

1.1 项目背景

现有桌面宠物程序 (`text1.c`) 基于 Windows GDI+ 分层窗口技术, 使用静态 PNG 图片的硬切换来模拟宠物动画。该方案存在以下局限:

- 动画效果生硬: 仅 9 张离散 PNG 整图切换, 无过渡帧、无插值
- 开发成本高: 每新增一个动作状态都需要绘制新 PNG
- 扩展性差: 难以实现 Live2D 风格的参数化变形

本方案旨在将桌面宠物迁移至 Unity 引擎, 利用 3D 模型渲染实现更流畅的动画效果, 并为后续引入 Live2D 技术奠定基础。

1.2 项目目标

1. 搭建基于 Unity 的桌面宠物框架, 支持透明窗口叠加在桌面上
2. 导入并渲染米哈游 (崩坏: 星穹铁道) 角色 3D 模型 (符玄)
3. 实现基础的桌面宠物交互功能: 拖拽、点击反馈、空闲动画
4. 保留现有程序的 AI 聊天功能 (通过进程间通信与 `text1.exe` 协作)
5. 为后续引入 Live2D Cubism SDK 预留接口

1.3 开发环境

- 引擎: Unity 2022.3.62t7 (LTS)
- 语言: C# (脚本)、HLSL (着色器)
- 平台: Windows 10/11 (Win64)
- 关键技术: TransparentWindowManager、Windows API (user32.dll)
- 源码管理: Git (仓库路径: `D:/Unity/projects/Desktop_per_pro`)

2 新旧架构对比

表 1: GDI+ 方案 vs Unity 方案对比

维度	旧方案 (GDI+)	新方案 (Unity)
渲染引擎	GDI+ (CPU 渲染)	Unity (DirectX 11/12, GPU 渲染)
模型格式	PNG 整图	FBX / AssetBundle (3D 网格)
动画方式	整图硬切换	骨骼动画 + Blend Shape
平滑度	无过渡 (硬切)	60fps 顶点插值
表情系统	无	Blend Shape 参数化
光照	预绘制在 PNG 中	实时 PBR 光照
物理模拟	简单重力公式	Unity Physics / 自定义
窗口透明	UpdateLayeredWindow (GDI)	TransparentWindowManager
鼠标交互	原生窗口消息	Unity Input System
模型复用	仅当前 9 张 PNG	可导入任意 3D 模型

3 技术架构

3.1 整体架构

新系统采用双层架构：Unity 负责 3D 渲染和交互，旧程序作为后台服务提供聊天和托盘功能。

```
1
2      Unity 窗口（透明叠加层）
3
4      Camera（Clear=SolidColor，BG=(0,0,0,0)）
5
6      符玄 3D 模型
7          空闲动画（Idle/Breath）
8          点击反应（Touch/Poke）
9          拖拽模式（OnMouseDown）
10         行走模式（Walk）
11         表情切换（BlendShape）
12
13     交互层
14         鼠标穿透区（透明区域穿透点击）
15         拖拽触发器（点击模型拖动）
16         碰撞体（Collider 检测点击）
17
18
19         进程间通信（UDP / 命名管道）
20
21
22     text1.exe（后台服务进程）
23     系统托盘图标
24     WebView2 AI 聊天窗口
25     设置窗口（权重/颜色配置）
26     托盘菜单（退出/隐藏）
27
```

3.2 透明窗口实现

利用 Unity_TransparentWindowManager 开源库 [1] 实现。核心原理：

1. 设置 Camera 的 ClearFlags 为 SolidColor，Background 为 (0,0,0,0)
2. 通过 Windows API SetWindowLong 设置 WS_EX_LAYERED 扩展样式
3. 调用 SetLayeredWindowAttributes 将颜色键 (0,0,0) 设为透明
4. 鼠标穿透：设置 WS_EX_TRANSPARENT 样式，让非模型区域点击穿透到桌面

3.3 模型渲染管线

1. 从社区获取符玄 FBX 模型（含骨骼和贴图）

2. 导入 Unity 作为 Model Prefab
3. 设置 Avatar 和 Animator Controller
4. 编写动画状态机: Idle ↔ Walk ↔ Touch ↔ Fall
5. 通过 Blend Shape 控制面部表情

3.4 进程间通信（可选）

若保留旧程序的 AI 聊天功能，Unity 和 `text1.exe` 之间可通过以下方式通信：

- UDP 本地回环：Unity 监听 127.0.0.1:xxxx，接收聊天命令
- 命名管道 (Named Pipe)：Windows 原生 IPC，更可靠
- 共享文件：最简单，但延迟较高

4 关键脚本设计

4.1 DesktopPet.cs — 宠物主控

负责宠物状态管理、物理模拟和生命周期。

```
1 using UnityEngine;
2
3 public class DesktopPet : MonoBehaviour
4 {
5     [Header("运动参数")]
6     public float gravity = 9.8f;           // 重力加速度
7     public float groundY = 0f;           // 地面 Y 坐标
8     public float moveSpeed = 2f;         // 行走速度
9     public float breathAmplitude = 0.02f; // 呼吸浮动幅度
10
11     // 宠物状态枚举
12     public enum PetState { Idle, Walking, Dragging, Falling, Landing
13         }
14
15     public PetState currentState = PetState.Idle;
16
17     private Vector3 velocity;             // 当前速度
18     private Animator animator;
19     private bool isGrounded = false;
20
21     void Start()
```

```
20     {
21         animator = GetComponent<Animator>();
22     }
23
24     void Update()
25     {
26         switch (currentState)
27         {
28             case PetState.Idle:
29                 UpdateIdle();
30                 break;
31             case PetState.Falling:
32                 UpdateFalling();
33                 break;
34             case PetState.Walking:
35                 UpdateWalking();
36                 break;
37         }
38     }
39
40     void UpdateIdle()
41     {
42         // 呼吸浮动（正弦波动）
43         float breath = Mathf.Sin(Time.time * 2f) * breathAmplitude;
44         transform.localPosition += Vector3.up * breath * Time.
            deltaTime;
45     }
46
47     void UpdateFalling()
48     {
49         // 重力下落
50         velocity.y -= gravity * Time.deltaTime;
51         transform.position += velocity * Time.deltaTime;
52
53         // 落地检测
54         if (transform.position.y <= groundY)
55         {
56             Vector3 pos = transform.position;
57             pos.y = groundY;
58             transform.position = pos;
59             velocity.y = 0;
```

```
60         isGrounded = true;
61         SwitchState(PetState.Landing);
62         // 播放落地动画后切回 Idle
63     }
64 }
65
66 void UpdateWalking()
67 {
68     // 随机左右移动
69     // 到达边界时掉头或停止
70 }
71
72 public void SwitchState(PetState newState)
73 {
74     currentState = newState;
75     // 触发对应动画
76     if (animator != null)
77         animator.SetInteger("State", (int)newState);
78 }
79 }
```

Listing 1: DesktopPet.cs 骨架

4.2 DragHandler.cs — 拖拽交互

处理鼠标拖拽宠物移动窗口的逻辑。

```
1 using UnityEngine;
2
3 public class DragHandler : MonoBehaviour
4 {
5     [Header("拖拽参数")]
6     public float throwVelocityScale = 5f; // 抛掷力度缩放
7
8     private Vector3 dragOffset;
9     private Vector3 lastMousePos;
10    private Vector3 throwVelocity;
11
12    void OnMouseDown()
13    {
14        // 记录点击偏移
15        dragOffset = transform.position - GetMouseWorldPos();
16        lastMousePos = GetMouseWorldPos();
```



```
17
18     // 通知 DesktopPet 进入拖拽状态
19     GetComponent<DesktopPet>()?.SwitchState(DesktopPet.PetState.
        Dragging);
20 }
21
22 void OnMouseDown()
23 {
24     Vector3 currentPos = GetMouseWorldPos();
25     // 计算抛掷速度
26     throwVelocity = (currentPos - lastMousePos) / Time.deltaTime;
27     lastMousePos = currentPos;
28
29     // 更新宠物位置
30     transform.position = currentPos + dragOffset;
31
32     // 同步更新 Unity 窗口位置 (调用 Win32 API)
33     UpdateWindowPosition();
34 }
35
36 void OnMouseUp()
37 {
38     // 松开时赋予初速度
39     DesktopPet pet = GetComponent<DesktopPet>();
40     if (pet != null)
41     {
42         // 将鼠标速度转换为物体的初速度
43         // pet.SetVelocity(throwVelocity * throwVelocityScale);
44         pet.SwitchState(DesktopPet.PetState.Falling);
45     }
46 }
47
48 Vector3 GetMouseWorldPos()
49 {
50     Vector3 mousePos = Input.mousePosition;
51     mousePos.z = Camera.main.WorldToScreenPoint(transform.
        position).z;
52     return Camera.main.ScreenToWorldPoint(mousePos);
53 }
54
55 void UpdateWindowPosition()
```

```

56     {
57         // 通过 Win32 API SetWindowPos 移动 Unity 窗口
58         // 使宠物始终跟随鼠标
59     }
60 }

```

Listing 2: DragHandler.cs 骨架

4.3 WindowOverlay.cs — 透明窗口管理

封装 Windows API 实现透明叠加和鼠标穿透。

```

1  using UnityEngine;
2  using System.Runtime.InteropServices;
3
4  public class WindowOverlay : MonoBehaviour
5  {
6      [DllImport("user32.dll")]
7      static extern int SetWindowLong(System.IntPtr hWnd, int nIndex,
8          int dwNewLong);
9
10     [DllImport("user32.dll")]
11     static extern int GetWindowLong(System.IntPtr hWnd, int nIndex);
12
13     [DllImport("user32.dll")]
14     static extern bool SetWindowPos(System.IntPtr hWnd, System.IntPtr
15         hWndInsertAfter,
16         int X, int Y, int cx, int cy, uint uFlags);
17
18     private const int GWL_EXSTYLE = -20;
19     private const int WS_EX_LAYERED = 0x80000;
20     private const int WS_EX_TRANSPARENT = 0x20;
21     private const int WS_EX_TOOLWINDOW = 0x80;
22
23     private static readonly System.IntPtr HWND_TOPMOST = new System.
24         IntPtr(-1);
25
26     private const uint SWP_NOMOVE = 0x0002;
27     private const uint SWP_NOSIZE = 0x0001;
28     private const uint SWP_SHOWWINDOW = 0x0040;
29
30     [Header("窗口设置")]
31     public bool enableClickThrough = false; // 是否启用鼠标穿透

```

```
29     void Start()
30     {
31         // 获取 Unity 窗口句柄并设置扩展样式
32 #if UNITY_STANDALONE_WIN
33         System.IntPtr hWnd = GetUnityWindowHandle();
34
35         // 设置分层 + 工具窗口（不在任务栏显示）
36         int exStyle = GetWindowLong(hWnd, GWL_EXSTYLE);
37         exStyle |= WS_EX_LAYERED | WS_EX_TOOLWINDOW;
38         SetWindowLong(hWnd, GWL_EXSTYLE, exStyle);
39
40         // 强制置顶
41         SetWindowPos(hWnd, HWND_TOPMOST, 0, 0, 0, 0,
42                     SWP_NOMOVE | SWP_NOSIZE | SWP_SHOWWINDOW);
43 #endif
44     }
45
46     void Update()
47     {
48         // 定时重新置顶（防止被任务栏遮挡）
49         if (Time.frameCount % 300 == 0)
50         {
51             ForceTopmost();
52         }
53     }
54
55     public void SetClickThrough(bool enabled)
56     {
57 #if UNITY_STANDALONE_WIN
58         System.IntPtr hWnd = GetUnityWindowHandle();
59         int exStyle = GetWindowLong(hWnd, GWL_EXSTYLE);
60         if (enabled)
61             exStyle |= WS_EX_TRANSPARENT;    // 添加穿透
62         else
63             exStyle &= ~WS_EX_TRANSPARENT;  // 移除穿透
64         SetWindowLong(hWnd, GWL_EXSTYLE, exStyle);
65         enableClickThrough = enabled;
66 #endif
67     }
68
69     void ForceTopmost()
```

```

70     {
71 #if UNITY_STANDALONE_WIN
72         System.IntPtr hWnd = GetUnityWindowHandle();
73         SetWindowPos(hWnd, HWND_TOPMOST, 0, 0, 0, 0,
74                     SWP_NOMOVE | SWP_NOSIZE);
75 #endif
76     }
77
78     System.IntPtr GetUnityWindowHandle()
79     {
80         // 通过 User32.dll 找到 Unity 窗口
81         return System.Diagnostics.Process.GetCurrentProcess().
82             MainWindowHandle;
83     }
84 }

```

Listing 3: WindowOverlay.cs 骨架

4.4 TouchController.cs — 点击反馈

检测鼠标点击模型时的反馈逻辑。

```

1 using UnityEngine;
2
3 public class TouchController : MonoBehaviour
4 {
5     [Header("点击反馈")]
6     public float reactionDuration = 1.0f; // 反应持续时间
7
8     private DesktopPet pet;
9     private float reactionTimer = 0f;
10    private bool isReacting = false;
11
12    void Start()
13    {
14        pet = GetComponent<DesktopPet>();
15    }
16
17    void OnMouseDown()
18    {
19        if (!isReacting)
20        {
21            // 触发点击反应

```

```
22         isReacting = true;
23         reactionTimer = reactionDuration;
24         pet.SwitchState(DesktopPet.PetState.Idle);
25
26         // 随机播放一个反应动画
27         Animator anim = GetComponent<Animator>();
28         if (anim != null)
29             anim.SetTrigger("Touch");
30
31         // 可选：播放音效
32     }
33 }
34
35 void Update()
36 {
37     if (isReacting)
38     {
39         reactionTimer -= Time.deltaTime;
40         if (reactionTimer <= 0f)
41         {
42             isReacting = false;
43             // 恢复正常状态
44         }
45     }
46 }
47 }
```

Listing 4: TouchController.cs 骨架

5 行走动画设计原理

版本：1.1

更新内容：补充 2D 侧面行走循环（Walk Cycle）动画理论，指导 Live2D 参数化走路动画的实现。

5.1 核心哲学

行走的本质是「摔倒—接住自己」的循环：重力不断将身体拉向地面，而双腿交替前伸，在身体即将倾倒的瞬间接住它。这一个循环中提炼出 4 个关键姿势（Key Poses），它们定义了行走的节奏和力学感。

5.2 四大关键姿势

表 2: 行走循环四大关键姿势

姿势	别名	描述	身体高度
Contact	跨步位、Heel & Toe	一脚跟触地、一脚尖离地， 身体处于「失重」瞬间	低
Down / Recoil	下落位、重量转移	重心完全压在前脚， 膝盖弯曲吸收冲击	最低
Passing	过地位	承重腿在身体正下方， 另一条腿经过其旁	最高
Up / High	抬高位、升空位	后腿推离地面， 前腿伸展准备落地	高

5.3 垂直弹跳相位

垂直弹跳是行走动画中**最重要的**视觉信号——没有它，行走就会变成「地面滑行」。

- **最高点**：Passing（过地位）——小腿肌肉将身体推起
- **最低点**：Down（下落位）——膝盖弯曲吸收冲击
- **相位序列**：Contact(低)→Down(**最低**)→Passing(**最高**)→Up(高)→Contact(对侧)
- **弹跳幅度**：自然行走约身高的 2-4%，卡通可放大到 10-20 cm

数学表达：设步态相位 $\phi \in [0, 2\pi)$ ，则

$$\text{BodyHeight}(\phi) = A \cdot \cos(\phi) + \text{BaseHeight}$$

(1)

其中 A 为弹跳幅度， $\phi = 0$ 时（Passing 位）身体最高， $\phi = \pi$ 时（Down 位）身体最低。

5.4 手臂与腿的交叉对位

核心法则：一条腿向前时，**对侧**手臂向前（同侧手臂向后）。

$$\text{Arm}_{\text{left}} = -\text{Leg}_{\text{right}} = +\text{Leg}_{\text{left}}$$

(2)

表 3: 手臂-腿交替关系

阶段	左腿	左臂	右臂
左腿在前 (Contact)	前	后	前
左腿过地 (Passing)	中	中	中
右腿在前 (Contact)	后	前	后

这一模式来自骨盆与肩膀的对向旋转，是人体自然步态的生物学基础。

5.5 身体前倾与速度

表 4: 行走类型与前倾角度

行走类型	前倾角度
休闲散步	3–7°
正常步行	5–10°
快走	10–20°
奔跑	20–30°+

前倾角度与速度呈正比：速度越快，需要更大的前倾来产生水平方向的动量。

5.6 头部运动

- 随身体弹跳：头上下的幅度约为身体弹跳的 50–70%
- 平滑补偿：大脑为保持视野水平，会使头部轨迹比骨盆更平滑
- 二次运动：头部可做微小延迟跟随（跟随主运动后到达），增加生动感
- 行进方向：朝行走方向微倾，自然行走时约低头 3–5°

5.7 Live2D 参数映射

结合符玄 Live2D 模型的实际可用参数，将行走循环映射如下：

表 5: 行走动画 → Live2D 参数映射

动画要素	Live2D 参数	典型值
身体转体（侧面视角）	ParamBodyAngleY	25°（恒定）
身体前倾	ParamBodyAngleX	5°（恒定）
脸转向侧面	ParamAngleY	25°（恒定）
低头看路	ParamAngleX	5°（恒定）
垂直颠簸	模型 Y 轴（像素偏移）	6 px
呼吸加深	ParamBreath	2.0（恒定偏大）
左腿参数		
抬腿	Param165 (抬腿)	±10°
前后摆动	Param126 (body X 腿动画位移)	±10°
弯曲透视	Param127 (body X 腿动画透视)	0–8°
右腿参数		
抬腿	Param164 (腿)	±10°（反向）
前后摆动	Param129 (body X 腿动画 R 位移)	±10°（反向）
弯曲透视	Param131 (body X 腿动画 R 透视)	0–8°（反向）
手臂摆动	Param94 (右手上臂)	±6°（对侧）
耸肩	Param153	0–3°

5.8 常见错误

- 1. 身体高度无变化（滑动步态 / Sliding）——最常见的错误，行走看起来像在地面滑行
- 2. 无重量转移——重心始终在中间，缺乏脚踏实地感
- 3. 同手同脚——混淆了交叉对位法则
- 4. 弹跳相位错误——最高点和最低点位置搞反，导致颠簸节奏违和
- 5. 头部刚性跟随——头应该比身体有轻微的延迟和独立运动
- 6. 速度与参数不协调——步频、步幅、前倾角度之间应保持一致性

6 实施路线

项目实施分为四个阶段，每阶段有明确的里程碑：

6.1 第一阶段：环境搭建（第 1 天）

- 在 Unity Hub 中创建新项目 DesktopPerPro
- 导入 TransparentWindowManager.unitypackage
- 配置 Camera 为透明背景
- 验证：运行后 Unity 窗口透明叠加在桌面上
- 初始化 Git 仓库，创建 .gitignore（Unity 模板）

6.2 第二阶段：模型导入（第 2–3 天）

- 获取符玄 3D 模型（FBX/Unity AssetBundle）
- 导入 Unity，配置贴图和材质
- 建立 Animator Controller，设置基本动画状态
- 搭建简单场景：将模型放置在透明背景中

6.3 第三阶段：交互实现（第 4–6 天）

- 实现 DesktopPet.cs：状态机 + 物理
- 实现 DragHandler.cs：拖拽 + 抛掷
- 实现 WindowOverlay.cs：透明 + 置顶
- 实现 TouchController.cs：点击反馈
- 测试：宠物可在桌面自由拖拽、下落、行走

6.4 第四阶段：集成与优化（第 7–10 天）

- 集成进程间通信（与旧 text1.exe 对话）
- 添加窗口置顶轮询（防止被遮挡）
- 性能优化（降低无关渲染开销）
- 打包为独立 Windows 可执行文件

7 模型获取方案

由于米哈游官方未直接公开角色模型文件，获取符玄 3D 模型有以下途径：

1. **SR-Model-Importer**: 社区开发的 Unity 工具，可从游戏资源包中提取角色模型
2. **hoyo-models 社区**: 部分站点提供已导出的 FBX 格式
3. **VRM 社区模型**: 有爱好者制作的 VRM 版本，可转换为 FBX
4. **替代方案**: 先用 Unity Asset Store 的免费角色模型调试交互逻辑，待模型到位后再替换

注意: 以上途径获取的模型仅限个人学习和研究使用，请勿用于商业用途或公开发布。

8 Live2D 过渡预留

为将来迁移至 Live2D 技术，在本项目中预留以下扩展点：

- **抽象渲染接口**: 将宠物渲染抽象为接口 `IPetRenderer`，当前实现为 3D 模型渲染，后续可替换为 Live2D Cubism SDK 渲染
- **参数化系统**: 当前使用 Unity Animator 的参数 (Float/Trigger)，与 Live2D 的参数系统 (ParamAngleX/ParamEyeLOpen 等) 概念一致，便于映射
- **交互逻辑复用**: 拖拽、点击、物理等交互逻辑与渲染方式解耦，更换渲染引擎时无需重写交互代码

```
1 // 预留的渲染接口，方便后续替换为 Live2D
2 public interface IPetRenderer
3 {
4     void SetParameter(string name, float value);
5     void PlayAnimation(string animationName);
6     void SetExpression(string expressionName);
7     void SetMouthOpen(float value); // 说话口型
8     void SetEyeOpen(float left, float right); // 眨眼
9 }
10
11 // Unity 3D 实现
12 public class UnityModelRenderer : MonoBehaviour, IPetRenderer
13 {
14     private Animator animator;
```

```

15     private SkinnedMeshRenderer meshRenderer;
16
17     public void SetParameter(string name, float value)
18     {
19         if (animator != null)
20             animator.SetFloat(name, value);
21     }
22
23     public void SetEyeOpen(float left, float right)
24     {
25         // 通过 Blend Shape 控制眼睛
26     }
27     // ... 其他接口实现
28 }
29
30 // 未来: Live2D 实现
31 // public class Live2DRenderer : MonoBehaviour, IPetRenderer { ... }

```

Listing 5: 渲染接口抽象（预留）

9 目录结构规划

```

Desktop_per_pro/           # Git 仓库根目录
|-- README.md              # 项目说明
|-- report.pdf             # 本文档（可删除）
|-- report.tex             # 本文档源码（可删除）
|-- Assets/
|   |-- Scenes/
|   |   +-- Main.unity    # 主场景
|   |-- Scripts/
|   |   |-- DesktopPet.cs
|   |   |-- DragHandler.cs
|   |   |-- WindowOverlay.cs
|   |   |-- TouchController.cs
|   |   |-- IPetRenderer.cs    # 渲染接口
|   |   +-- UnityModelRenderer.cs
|   |-- Models/
|   |   +-- FuXuan/          # 符玄模型
|   |-- Materials/
|   |-- Animations/

```

```
|      |      +-- AnimatorController.controller
|      +-- Plugins/
|      +-- TransparentWindowManager/
|-- Packages/
|-- ProjectSettings/
+-- .gitignore
```

参考文献

- [1] XJINE. *Unity_TransparentWindowManager — Make Unity's window transparent and overlay on desktop.* https://github.com/XJINE/Unity_TransparentWindowManager, 2018. MIT License.
- [2] hoyo-models Community. *Honkai: Star Rail 3D Character Models.* <https://hoyo-models.vercel.app/hsr>, 2025.
- [3] lethern. *SR-Model-Importer — Honkai: Star Rail Model Importer for Unity.* <https://github.com/lethern/SR-Model-Importer>, 2025.
- [4] 原桌面宠物项目. *Desktop Pet — GDI+ + WebView2 桌面宠物.* 路径:D:/C/Desktop pet/text1.c, 2025.